

TradeBench — Plain-language explainer

What the task is, what the agent learns, how reward and long-horizon work

April 26, 2026

1. What is the task, in one paragraph

TradeBench is a **simulated trading game** where a language-model agent is given \$100,000 and a basket of 10 stocks, and asked to manage the portfolio for a year of trading days (one decision per day). At the end of the year we measure how well it did — not against a fixed correct answer, but against what a passive “hold everything equally” investor would have earned over the same period. The agent has to decide, every day: *what is happening in the market right now, and given what I know, should I buy, sell, hold, or change weights?* It writes that decision down, may place orders, then the clock advances to the next trading day. After 120 days, the run ends.

2. What are we teaching the model?

We are not teaching it the “right answer” — there is none. We are teaching it **how to reason about uncertain financial outcomes over many sequential steps**. Specifically:

- **Plan from data:** read a year’s worth of historical prices, write a strategy, then commit to it.
- **Stay disciplined:** don’t run leverage, don’t concentrate everything in one stock, don’t churn your portfolio.
- **Beat the market:** an investor who held everything equally is the benchmark. If the agent only matches it, that’s mediocre. If the agent beats it, that’s real skill (alpha).
- **Execute, not just talk:** it’s not enough to *write down* a strategy — the agent must actually translate that strategy into trades. (This is the lever the reflection loop is currently optimizing.)
- **Don’t cheat:** no peeking at future prices, no using its memorized knowledge of how 2024 actually unfolded, no `eval/exec` shenanigans inside the sandbox.

Think of it like a trading internship: the model is not being graded on whether it knows the answers, but on whether it can analyze data, articulate a thesis, and execute it consistently across hundreds of decisions in a row.

3. What does the data look like?

3.1 The universe

Ten fictional assets called `tier_a01`, `tier_a02`, ..., `tier_a10`. Each is backed by **real historical price data** from a real publicly-traded company (TXN, ORCL, AAPL, WMT, CSCO, JPM, XOM, PEP, TSLA, DIS — but the model only sees the alias names, never the real tickers). This way the model cannot use memorized knowledge of how those companies actually performed.

3.2 Two phases

There are **two phases** in every rollout:

Phase 1 — TRAIN (the “research” phase, 1 decision)

The agent gets a single one-shot view of a full year of historical price data:

- Per-asset summary statistics (mean return, volatility, Sharpe ratio, max draw-down, total return)
- A 10×10 correlation matrix
- The first 5 and last 5 close prices for each asset

Looking at all of this, the agent must produce **one strategy summary** in JSON form:

```
regime_label:      "sharpe_weighted_diversified"
intended_exposure: 0.80    (use 80% of capital, hold 20% as cash)
top_convictions:   [tier_a03 weight 0.20, tier_a07 weight 0.20, ...]
edge_summary:      "Top 5 by Sharpe ratio, equal-weighted, monthly rebalance"
reasoning:         "..."
```

That’s it for train phase. No buying, no selling, no time advancing. The model writes a one-page hedge-fund pitch and the phase ends.

Phase 2 — TEST (the “execution” phase, 120 trading days)

A 120-bar walk-forward rollout starting **after** the train window ends. Each bar (= one trading day):

1. The agent can look at its current portfolio (`view_portfolio`).
2. The agent can run a Python sandbox to compute fresh signals (`sandbox_exec`) — but only on past data, never future.
3. The agent **must** call `record_decision` to articulate its current view (regime + convictions + intended exposure).
4. The agent can place buy/sell orders (`place_order`), each one queued for execution at the next-day open.
5. The agent calls `advance_day`, which advances the clock by one trading day. Orders execute, prices update, a per-bar reward is computed.

Important constraint: `advance_day` **is rejected** if the agent skipped step 3 — every bar requires a fresh decision. This is the long-horizon gate.

The strategy committed during the train phase is fed back into the test phase as a soft anchor — the agent sees its own train-phase commitment in every test-bar prompt, and can choose to follow it, modify it, or override it as new evidence arrives.

4. The reward function — what makes the model “good”?

The reward is a number between 0 and 1 emitted on every bar. The cumulative score across 120 bars (also bounded 0 to 1 when divided by the bar count) is what we plot. Here is how it is computed:

4.1 The formula in plain language

For each bar, we score the agent on **seven independent qualities**, then take a weighted average. If any rule is violated, the entire bar’s reward is zeroed out.

```
reward = (40% × alpha vs market)
        + (15% × absolute return)
        + (10% × drawdown discipline)
        + (10% × distance from ruin)
        + (10% × turnover discipline)
        + (5% × diversification discipline)
        + (10% × consistency of outperformance)
        - multiplied by -
        (1 if no rules broken, else 0)
```

Each of the seven components is itself a number between 0 and 1. The weights add up to 1, so the weighted sum is naturally between 0 and 1. Multiplying by the rules-gate just zeros out cheating.

4.2 What each component measures

c_alpha — beating the market (40% weight, the dominant signal)

We compute what an equal-weight buy-and-hold investor (the **benchmark**) would have earned by now. We compare the agent’s portfolio against that benchmark. If the agent is ahead, this score climbs above 0.5 toward 1.0. If the agent is behind, it falls below 0.5 toward 0.

This is the only component that can really distinguish *skill* from *luck*. In a bull market, both the agent and the benchmark go up — the question is whether the agent goes up *faster*.

c_return — making money in absolute terms (15% weight)

A simple sigmoid of the agent’s cumulative log-return. Flat = 0.5, profit = above 0.5, loss = below 0.5. This rewards the agent for growing capital even when there is no benchmark to compare against.

c_drawdown — not losing too much from peak (10% weight)

Tracks “how far below your high-water mark are you today?” A 10% drawdown drops this score to 0.8; a 50% drawdown drops it to 0. Linear, not quadratic — every percent of drawdown hurts equally.

c_solvency — distance from ruin (10% weight)

A sigmoid centered at 50% of starting capital. If you’re well above \$50K (half of \$100K starting), you score near 1.0. If you’re approaching \$50K, the score drops sharply. Bankruptcy → 0.

c_efficiency — not churning (10% weight)

Soft penalty on turnover. If you trade less than 10% of portfolio per bar, full credit. Beyond that, the score decays exponentially — this discourages frantic re-trading without good reason.

c_diversity — not betting it all on one stock (5% weight)

Based on the **Herfindahl Index** (a concentration measure). 10 equal-weight names = max diversity → 1.0. All in one stock = 0.0.

c_consistency — outperformance with low drawdown variance (10% weight)

A “downside Sortino” — only kicks in when the agent is *already* above the benchmark. Discourages roller-coaster paths even on winning runs. Sub-benchmark agents get a flat 0.5 regardless of volatility (otherwise they’d get free credit for staying flat at zero).

g_compliance — the rules gate (multiplicative, {0, 1})

Three things zero this out instantly:

- **Rules violation** (regex catches the agent claiming memorized facts about specific 2024 events).
- **Reward-hack violation** (regex catches `eval`, `exec`, `sandbox-escape` patterns).
- **Leverage breach** (gross leverage > 1.0 — long-only env so this should never fire).

If any of those is true on a bar, the bar’s reward is zero, no matter how well the other 7 components scored.

4.3 Why this design, in one paragraph

The old reward had a saturating Sharpe-bonus term that hit its cap of 0.2 within ~5 days for **any** portfolio with positive mean return. That meant a buy-and-hold portfolio and a properly-managed alpha portfolio both got roughly the same score, and the reward was **not differentiating skill**. The new design fixes this with three properties: **(1)**

every component is bounded $[0, 1]$ by sigmoid or linear clipping, no saturation cliffs; **(2)** the weights are a convex combination so the total is naturally in $[0, 1]$ without any post-hoc normalization; **(3)** the dominant signal (40% weight) is **alpha vs the equal-weight benchmark** — the only thing that can genuinely tell apart skill from market beta.

A typical “disciplined but unskilled” agent (holds positions, doesn’t trade much, doesn’t lose money but doesn’t gain much either) scores around 0.61. A do-nothing all-cash agent scores around 0.54 in a bull market. An alpha-capture agent that beats the benchmark by ~10 percentage points scores around 0.75. A catastrophic-loss-with-violations agent scores ~0.05.

5. How is this a “long-horizon” task?

A long-horizon task is one where a single rollout requires the agent to make **many sequential decisions, each depending on the previous ones**. Specifically:

5.1 The mechanics

- **120 trading days** per test rollout (~6 months of real-world calendar time, compressed into a 25-minute simulation).
- **Per-bar decision gate:** the agent must call `record_decision` *every single bar* before the clock advances. The environment hard-rejects `advance_day` if no decision was recorded for the current date. This is enforced in the env, not just the prompt.
- **Cumulative state:** the agent’s portfolio, cash, and high-water-mark all carry from one bar to the next. A bad decision on bar 12 affects available cash on bar 13, which affects what trades are possible on bar 14, and so on.
- **Carry strategy:** the agent’s train-phase strategy is shown in every test-bar prompt, so the agent has to either follow it consistently or articulate why it’s deviating (each bar).

5.2 The numbers

A typical 120-bar rollout produces:

Action	Per bar	Per rollout
<code>record_decision</code>	~1.2 (mandatory ≥ 1)	~140
<code>advance_day</code>	~1.3 (some gate-rejected)	~150
<code>sandbox_exec</code> (Python analysis)	~0.2	~22
<code>view_portfolio</code>	~0.1	~6
<code>place_order</code>	~0.05	~6
Total LLM calls	~2.7	~320

That’s ~**320 sequential LLM calls per single rollout**, with each call seeing the full history of prior decisions in its prompt. The reflection loop runs 5–6 of these rollouts in

series, so a full optimization run is ~**1,600-2,000 sequential LLM calls** all working toward a single objective (beat the market while staying disciplined).

5.3 Why this is hard

A short-horizon task is something like “summarize this article” — one input, one output, done in one call. A long-horizon task forces the model to:

- **Maintain a coherent thesis across hundreds of decisions** without contradicting itself.
- **Adapt to new information** (each new bar’s price moves are feedback on the prior decision).
- **Handle the credit-assignment problem** — if the portfolio is up 5% on bar 100, *which* of the 250 prior decisions actually drove that gain?
- **Resist drift** — it’s easy to gradually shift from “balanced long” to “all-in on one stock” through many small decisions; the model has to actively notice and resist that.

Empirically, even strong frontier models tend to have specific failure modes here: they get stuck repeating the same regime label every bar, they stop placing orders even though their committed strategy says they should, they over-trade in volatile periods, or they freeze entirely and just `advance_day` after a one-line decision. **The reflection loop is what catches and fixes these by rewriting the model’s system prompt between rollouts**, using the full record of what the model said and did during the prior rollout.

6. The big picture — what is being optimized

There are three things to keep separate:

Thing	What it is	Who optimizes it
The agent (Qwen3-32B / GLM-5.1)	Walks the 120-bar rollout, makes decisions	Not optimized — runs as a frozen model
The system prompt	Instructions given to the agent	The reflection loop (Claude Opus 4.7)
The reward function	How we score what the agent did	Hand-designed (the 7-component formula above)

Each rollout is a noisy measurement of *how well a given system prompt drives the agent*. The reflection loop reads the rollout (full reasoning + every action + every reward component), proposes a surgical edit to the prompt, and the next rollout measures whether that edit helped. After 5 iterations we should see a clear improvement trajectory: baseline → iter 1 → iter 2 → ... → iter 5, with `score_normalized` climbing each time.

This is **GEPA-style reflective prompt optimization**: instead of fine-tuning the model weights (expensive, requires labeled gradients), we hold the model fixed and optimize the *instructions*. The model is a tool; the prompt is the tool-handle that the optimizer adjusts.

7. In one sentence

We give a language model a fictional brokerage account and a year of price data, force it to make a fresh decision every single trading day for 120 days in a row, score every bar on seven different “is this a good portfolio managed well?” criteria that all live in $[0, 1]$, and use a stronger model to read the full transcript and rewrite the system prompt between rollouts so the agent gets better at the task without any weight updates.